INSTITUTO MAUÁ DE TECNOLOGIA
CENTRO UNIVERSITÁRIO
ENGENHARIA DE COMPUTAÇÃO

DAVID SANCHEZ BITTENCOURT DANIEL — RA: 24.00029-9
KAUÃ BUENO MARCATTO ORTEGA — RA: 24.00031-0

GRAFOS E SUAS APLICAÇÕES: FUNDAMENTOS, ALGORITMOS E ESTUDO DE CASO NA CONTENÇÃO DE DESINFORMAÇÃO EM REDES SOCIAIS

Implementação de rastreamento em ondas com Busca em Largura e mitigação de Fake News por Quarentena e Pontos de Articulação

SÃO CAETANO DO SUL
2026

DAVID SANCHEZ BITTENCOURT DANIEL
KAUÃ BUENO MARCATTO ORTEGA

GRAFOS E SUAS APLICAÇÕES: FUNDAMENTOS, ALGORITMOS E ESTUDO DE CASO NA CONTENÇÃO DE DESINFORMAÇÃO EM REDES SOCIAIS

Trabalho acadêmico semestral apresentado à disciplina ECM306 — Tópicos Avançados de Estrutura de Dados do Instituto Mauá de Tecnologia, como requisito parcial para avaliação do 2º semestre.

Orientador: Prof. José Calvetti.

SÃO CAETANO DO SUL
2026

RESUMO

Este trabalho apresenta um estudo aprofundado sobre a Teoria dos Grafos, abrangendo desde os seus fundamentos matemáticos e representações computacionais até estruturas clássicas e paradigmas de projeto de algoritmos. Revisam-se propriedades estruturais como graus de vértices, lema do aperto de mãos, conexidade, grafos eulerianos e hamiltonianos, emparelhamentos em grafos bipartidos com o Teorema de Hall, planaridade com o Teorema de Kuratowski e coloração. No campo algorítmico, detalham-se os métodos de Busca em Largura (BFS), Busca em Profundidade (DFS), o algoritmo de Dijkstra para caminhos mínimos e o algoritmo de Kahn para ordenação topológica. Com base nesse arcabouço, desenvolve-se uma solução computacional completa e original em Java puro (JDK 21) voltada à simulação, rastreamento e contenção da disseminação de desinformação (fake news) em grupos de mensagens. A propagação do boato a partir de um paciente zero é modelada por ondas discretas via BFS em complexidade $O(V + E)$. Para a mitigação, implementa-se um algoritmo exato de quarentena por força bruta em $O(V \cdot (V + E))$, comparando seu resultado com a heurística gulosa de maior grau e com o algoritmo de Tarjan para identificação de pontos de articulação em $O(V + E)$. Os experimentos práticos evidenciam um fenômeno didático central: o vértice de maior grau nem sempre é a intervenção ótima, sendo superado por vértices com menor conectividade que atuam como pontes estruturais entre comunidades. Uma interface interativa baseada na estética de quadro investigativo valida a viabilidade operacional e didática da proposta.

Palavras-chave: Teoria dos Grafos; Busca em Largura; Fake News; Quarentena; Pontos de Articulação; Algoritmo de Tarjan.

ABSTRACT

This work presents an in-depth study of Graph Theory, spanning from mathematical foundations and computational representations to classical structures and algorithm design paradigms. Structural properties such as vertex degrees, the handshaking lemma, connectivity, Eulerian and Hamiltonian graphs, matchings in bipartite graphs with Hall's Marriage Theorem, planarity with Kuratowski's Theorem, and graph coloring are comprehensively reviewed. Regarding algorithms, Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's shortest path, and Kahn's topological sorting are analyzed. Grounded in this framework, a complete and original computational solution is developed in pure Java (JDK 21) focused on simulating, tracking, and containing misinformation (fake news) spreading across messaging groups. Spreading from a patient zero is modeled as discrete waves via BFS in $O(V + E)$ time. For mitigation, an exact brute-force quarantine algorithm in $O(V \cdot (V + E))$ is implemented and contrasted against both the highest-degree greedy heuristic and Tarjan's bridge-finding algorithm in $O(V + E)$. Practical experiments demonstrate that the highest-degree node is not always the optimal quarantine target, often superseded by lower-degree vertices acting as structural bridges between communities. An interactive web interface styled as an investigative board confirms the operational and pedagogical validity of the approach.

Keywords: Graph Theory; Breadth-First Search; Misinformation; Quarantine; Articulation Points; Tarjan's Algorithm.

LISTA DE ILUSTRAÇÕES

Figura 1 – Grafos não planares canônicos de Kuratowski: K_5 e $K_{3,3}$

Figura 2 – Emparelhamento máximo em grafo bipartido e condição de Hall

Figura 3 – Modelo de propagação em ondas da Busca em Largura (BFS)

Figura 4 – Interface do software 'Caso Fake News: Operação Grafos' em execução

Figura 5 – Benchmark de desempenho da análise de quarentena por força bruta

LISTA DE TABELAS

Tabela 1 – Comparação de complexidade: Lista vs. Matriz de Adjacência

Tabela 2 – Síntese comparativa entre trilhas eulerianas e ciclos hamiltonianos

Tabela 3 – Mapeamento de problemas cotidianos e estruturas de grafos

Tabela 4 – Rastreamento da fila BFS na rede de teste do estudo de caso

Tabela 5 – Ranking de candidatos à quarentena: Força Bruta vs. Maior Grau

Tabela 6 – Benchmark de tempo de execução da força bruta para diferentes ordens de V

SUMÁRIO

1 INTRODUÇÃO 7

2 FUNDAMENTOS TEÓRICOS DE GRAFOS 8

2.1 Definições Formais e Grau de Vértices 8

2.2 Representações Computacionais em Memória 9

2.3 Caminhos, Ciclos e Conexidade 10

3 ESTRUTURAS ESPECIAIS E PROPRIEDADES CLÁSSICAS 11

3.1 Grafos Eulerianos e Hamiltonianos 11

3.2 Emparelhamentos Bipartidos e Teorema de Hall 12

3.3 Planaridade e Coloração de Grafos 13

4 ALGORITMOS FUNDAMENTAIS E PARADIGMAS DE PROJETO 14

4.1 Buscas em Grafos: BFS e DFS 14

4.2 Caminhos Mínimos e Ordenação Topológica 15

4.3 Técnicas de Projeto: Força Bruta, Gulosos e Programação Dinâmica 16

5 PANORAMA DE APLICAÇÕES COMPUTACIONAIS 17

6 ESTUDO DE CASO: RASTREAMENTO E QUARENTENA DE FAKE NEWS 18

6.1 Formulação do Problema e Perguntas de Modelagem 18

6.2 Modelagem Formal e Estrutura dos Dados 19

6.3 Propagação em Ondas via Busca em Largura (BFS) 20

6.4 Mitigação Exata por Força Bruta vs. Heurística de Grau 21

6.5 Diagnóstico Estrutural por Pontos de Articulação (Tarjan) 22

6.6 Arquitetura de Software e Interface com o Usuário 23

6.7 Resultados Experimentais e Análise de Escalabilidade 24

7 CONCLUSÃO 25

REFERÊNCIAS 26

1 INTRODUÇÃO

A Teoria dos Grafos constitui um dos ramos mais férteis e aplicados da Ciência da Computação e da Matemática Discreta. Surgida formalmente em 1736 com a resolução do célebre problema das Pontes de Königsberg por Leonhard Euler, a disciplina evoluiu de uma curiosidade topológica para a linguagem fundamental de modelagem de sistemas complexos, redes de transporte, circuitos eletrônicos, estruturas moleculares e mídias sociais contemporâneas.

No ambiente digital moderno, a interconexão ubíqua propiciada por aplicativos de mensagens instantâneas estabeleceu redes de relacionamento de altíssima capilaridade. Se, por um lado, tal infraestrutura potencializa a circulação veloz de dados, por outro, propicia o fenômeno deletério da disseminação em massa de desinformação (fake news). A propagação viral de boatos opera de maneira estritamente relacional: uma informação falsa inserida por um emissor inicial alcança seus contatos imediatos, os quais, ao retransmiti-la, deflagram ondas subsequentes de contágio informacional.

Diante desse panorama, o presente trabalho atende às diretrizes da disciplina ECM306 — Tópicos Avançados de Estrutura de Dados, desenvolvendo uma pesquisa abrangente intitulada 'Grafos e suas Aplicações' e implementando um projeto autoral sob o lema 'Resolvendo um Problema com Grafo'. Rejeitando formulações clichês baseadas em sistemas de navegação por GPS ou mapas viários, o estudo propõe a formalização matemática e algorítmica do problema de contenção de boatos em redes de mensagens.

O trabalho estrutura-se em sete capítulos: os Capítulos 2 a 4 cobrem com rigor os conceitos fundamentais, estruturas clássicas e famílias algorítmicas lecionadas pelo Prof. Calvetti; o Capítulo 5 compila um panorama abrangente de aplicações práticas no cotidiano; o Capítulo 6 detalha a engenharia do estudo de caso, formalizando o modelo, as classes Java, a simulação em ondas por BFS, a mitigação exata por força bruta em contraste com heurísticas de grau e os pontos de articulação de Tarjan; por fim, o Capítulo 7 sintetiza as conclusões obtidas.

2 FUNDAMENTOS TEÓRICOS DE GRAFOS

2.1 Definições Formais e Grau de Vértices

Formalmente, um grafo G é definido como um par ordenado G = (V, E), no qual V = {v1, v2, ..., vn} representa um conjunto não vazio de elementos denominados *vértices* (ou nós), e E = {e1, e2, ..., em} denota um conjunto de *arestas* (ou arcos), sendo cada aresta e ∈ E associada a um par não ordenado (ou ordenado, no caso de digrafos) de vértices {u, v} ⊆ V (NICOLETTI; HRUSCHKA, 2017).

Dois vértices u e v unidos por uma aresta são ditos adjacentes ou vizinhos. O conjunto de vizinhos de um vértice v é denominado sua vizinhança aberta, denotada por N(v). O grau de um vértice v, expresso por d(v) ou grau(v), corresponde ao número de arestas incidentes em v. Um vértice é classificado como par se d(v) for par, e ímpar caso contrário. Um resultado fundamental da teoria clássica é o Teorema da Soma dos Graus, frequentemente chamado de *Lema do Aperto de Mãos* (Handshaking Lemma):

Teorema 1 (Soma dos Graus). Seja G = (V, E) um grafo não direcionado com n vértices e m arestas. Então:

$$\sum_{v \in V} d(v) = 2 \cdot |E|$$

Demonstração. Cada aresta e = {u, v} conecta exatamente dois vértices, contribuindo com uma unidade no cômputo de d(u) e uma unidade em d(v). Consequentemente, ao somar os graus de todos os vértices de V, cada aresta é contabilizada exatamente duas vezes. Q.E.D.

Corolário 1. Em todo grafo não direcionado, a quantidade de vértices de grau ímpar é obrigatoriamente par. De fato, se dividirmos a soma dos graus em vértices pares e ímpares, a soma total é par (2|E|); como a soma dos pares é par, a soma dos ímpares também deve ser par, o que só ocorre se houver um número par de parcelas ímpares.

2.2 Representações Computacionais em Memória

A escolha da estrutura de dados para armazenar G em memória impacta criticamente a eficiência temporal dos algoritmos (CORMEN et al., 2012). As duas abordagens canônicas são a *Matriz de Adjacência* e a *Lista de Adjacência*:

• **Matriz de Adjacência:** matriz bidimensional M de dimensões |V| × |V|, onde M[u][v] = 1 se {u, v} ∈ E e 0 caso contrário. Possui custo de memória quadrático $O(V^2)$, permitindo consultar a existência de arestas em tempo O(1), mas penalizando a iteração sobre a vizinhança de um nó em O(V). É indicada primordialmente para grafos densos, onde |E| se aproxima de $|V|^2$.

• **Lista de Adjacência:** arranjo de |V| listas encadeadas ou dinâmicas, onde a lista associada ao vértice u armazena todos os seus vizinhos N(u). Consome espaço proporcional a O(V + E), viabilizando enumerar vizinhos em tempo ótimo O(grau(u)). É a representação padrão para grafos esparsos, como redes de telecomunicações e redes sociais.

Tabela 1 – Comparação de complexidade: Lista vs. Matriz de Adjacência

Operação / Critério	Lista de Adjacência	Matriz de Adjacência
Espaço em Memória	O(V + E) — Ótimo p/ esparsos	O(V²) — Excessivo p/ esparsos
Consultar aresta (u, v)	O(d(u))	O(1) — Acesso direto
Listar vizinhos de u	O(d(u)) — Imediato	O(V) — Varredura completa
Adicionar vértice	O(1) amortizado	O(V²) — Realocação
Perfil mais indicado	Redes sociais, web, esparsas	Grafos densos, algébricos

Fonte: Adaptado de Cormen et al. (2012) e Nicoletti e Hruschka (2017).

2.3 Caminhos, Ciclos e Conexidade

A formalização das sequências de vértices e arestas é indispensável para a teoria algorítmica:

• **Passeio (Walk):** sequência alternada v0, e1, v1, e2, ..., vk de vértices e arestas conectadas.

• **Trilha (Trail):** passeio no qual todas as arestas são distintas duas a duas.

• **Caminho (Path):** trilha na qual todos os vértices são distintos (com exceção eventual do início e término).

• **Ciclo:** caminho fechado de comprimento $k \geq 3$, cuja origem coincide com o destino.

Um grafo é dito *conexo* se existir ao menos um caminho entre qualquer par de seus vértices. Quando não é conexo, ele se decompõe em subgrafos conexos maximais denominados *componentes conexos* (BOAVENTURA NETTO, 2014).

3 ESTRUTURAS ESPECIAIS E PROPRIEDADES CLÁSSICAS

3.1 Grafos Eulerianos e Hamiltonianos

A distinção entre cobrir todas as conexões ou visitar todas as entidades constitui uma das fronteiras conceituais mais relevantes da teoria computacional:

• **Grafo Euleriano:** grafo conexo que admite um circuito fechado contendo cada aresta de E exatamente uma vez. Pelo Teorema de Euler (1736), um grafo conexo é euleriano se e somente se todos os seus vértices possuírem grau par. Se admitir exatamente dois vértices de grau ímpar, existe uma trilha euleriana aberta iniciando em um ímpar e terminando no outro.

• **Grafo Hamiltoniano:** grafo que possui um ciclo simples fechado que visita cada vértice de V exatamente uma vez. Diferentemente do caso euleriano, não se conhece um critério estrutural simples e de verificação polinomial que caracterize todos os grafos hamiltonianos; as condições clássicas conhecidas (Teorema de Dirac: $d(v) \geq n/2$; Teorema de Ore: $d(u)+d(v) \geq n$) são apenas suficientes. A decisão de hamiltonicidade é um problema NP-completo clássico.

Tabela 2 – Síntese comparativa entre trilhas eulerianas e ciclos hamiltonianos

Critério	Problema Euleriano	Problema Hamiltoniano
Objeto a ser coberto	Todas as arestas (E)	Todos os vértices (V)
Repetição de vértices	Permitida ao longo da trilha	Proibida (exceto retorno à origem)
Critério de existência	Todos os graus pares (exato)	Condições parciais (Dirac, Ore)
Complexidade de decisão	Polinomial P — Alg. Hierholzer O(E)	NP-completo — Explosão fatorial
Exemplo prático	Varrição de ruas, leitura de hidrômetros	Visita a clientes (Caixeiro Viajante)

Fonte: Elaborado pelos autores a partir do material de aula (Calvetti, 2026).

3.2 Emparelhamentos Bipartidos e Teorema de Hall

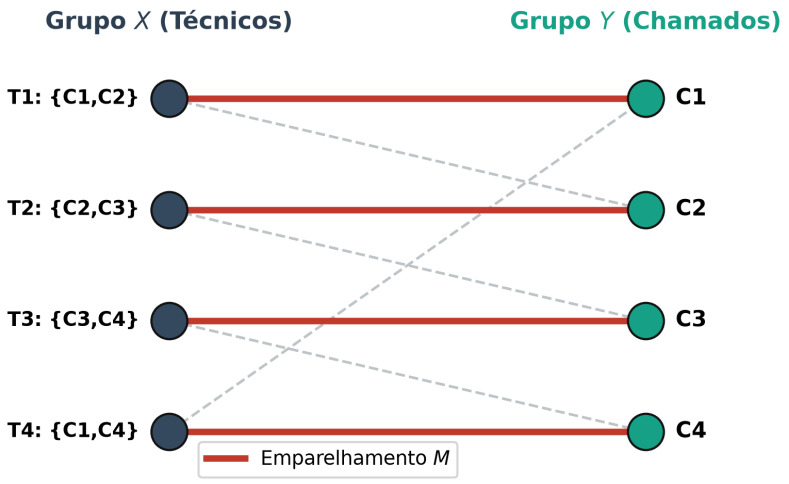
Um grafo $G = (V, E)$ é *bipartido* se V puder ser particionado em dois subconjuntos disjuntos X e Y tais que toda aresta em E ligue um elemento de X a um de Y. Um conjunto $M \subseteq E$ é dito um *emparelhamento* (matching) se duas arestas quaisquer de M não compartilharem vértices em comum. Diz-se que M satura o conjunto X se todo vértice $x \in X$ for extremidade de alguma aresta de M.

Teorema de Hall (Philip Hall, 1935). Seja G um grafo bipartido com bipartição (X, Y). Existe um emparelhamento que satura X se, e somente se, para todo subconjunto $S \subseteq X$, a cardinalidade de sua vizinhança satisfizer:

$$|\mathbf{N(S)}| \geq |\mathbf{S}|$$

A condição de Hall formaliza a barreira fundamental de alocação: se um subconjunto de solicitantes S demandar mais parceiros do que o total conjunto de opções que lhes são compatíveis (|N(S)| < |S|), o atendimento pleno torna-se estruturalmente impossível. Pelo Teorema de Berge (1957), um emparelhamento é máximo se e somente se não contiver caminhos M-aumentantes.

Figura 2 – Emparelhamento máximo em grafo bipartido e condição de Hall



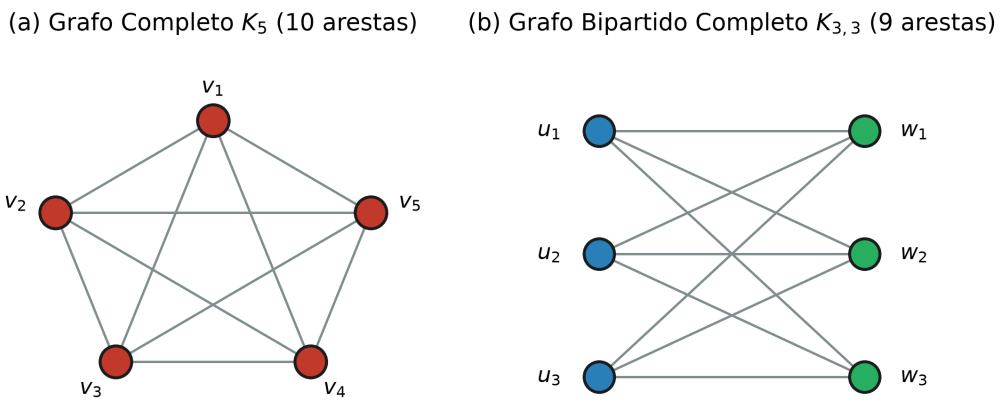
Fonte: Elaborado pelos autores (2026).

3.3 Planaridade e Coloração de Grafos

Um grafo é dito *plano* se estiver desenhado sobre uma superfície bidimensional de modo que suas arestas não se interceptem a não ser em suas extremidades. Um grafo é *planar* se for isomorfo a um grafo plano. Para qualquer grafo planar conexo, vale a *Fórmula Poliedral de Euler*: **|V| - |E| + |F| = 2**, onde F representa o conjunto de faces (regiões delimitadas).

Teorema de Kuratowski (1930). Um grafo G é planar se, e somente se, não contiver nenhum subgrafo homeomorfo (obtido por subdivisão de arestas) aos grafos canônicos K5 (grafo completo de 5 vértices) ou K3,3 (grafo bipartido completo das 3 casas e 3 serviços).

Figura 1 – Grafos não planares canônicos de Kuratowski: K_5 e $K_{3,3}$



Fonte: Elaborado pelos autores (2026).

A planaridade conecta-se diretamente ao clássico *Problema da Coloração de Vértices*: atribuir cores aos vértices de modo que nós adjacentes recebam cores distintas. Pelo célebre *Teorema das Quatro Cores* (Appel & Haken, 1976), qualquer grafo planar pode ser colorido com no máximo 4 cores. Em grafos gerais, encontrar o número cromático $\chi(G)$ é NP-difícil, demandando algoritmos gulosos de ordenação de graus como Welsh-Powell e DSatur.

4 ALGORITMOS FUNDAMENTAIS E PARADIGMAS DE PROJETO

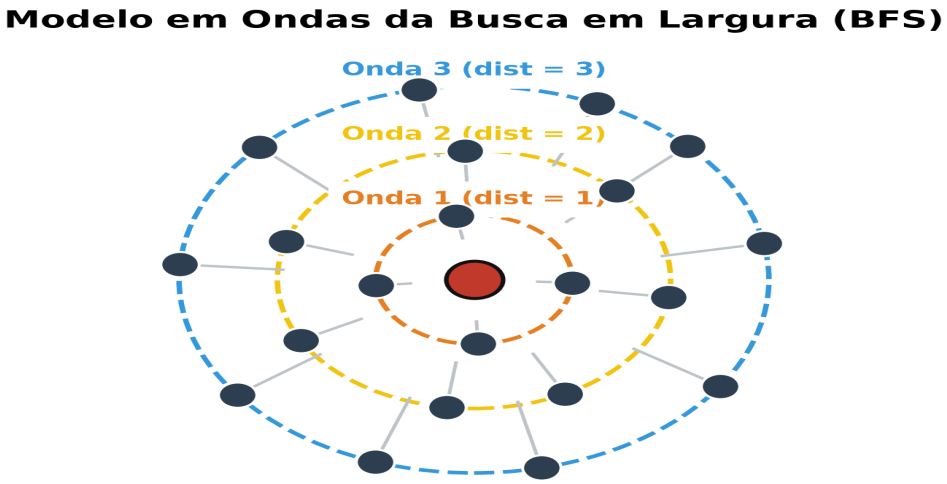
4.1 Buscas em Grafos: BFS e DFS

As estratégias de travessia sistemática formam a espinha dorsal dos algoritmos sobre grafos:

• **Busca em Largura (BFS - Breadth-First Search):** explora a rede em 'ondas concêntricas' utilizando uma fila FIFO (First-In, First-Out). A partir de uma raiz s, todos os vértices à distância 1 são visitados antes dos vértices à distância 2, garantindo que o caminho encontrado seja o de menor número de arestas em grafos não ponderados. O custo assintótico é O(V + E).

• **Busca em Profundidade (DFS - Depth-First Search):** avança o mais profundamente possível ao longo de cada ramificação antes de retroceder (backtracking), operando sobre uma pilha LIFO (ou pilha de recursão). Permite classificar as arestas em árvore, retorno, avanço e cruzamento, constituindo a base para detecção de componentes fortemente conexos e pontos de corte.

Figura 3 – Modelo de propagação em ondas da Busca em Largura (BFS)



Fonte: Elaborado pelos autores (2026).

4.2 Caminhos Mínimos e Ordenação Topológica

Quando as arestas possuem pesos não negativos representando tempos, custos ou distâncias, a BFS deixa de garantir a otimização, exigindo o algoritmo guloso de **Dijkstra (1959)**. Mantendo uma fila de prioridades (PriorityQueue), o algoritmo extrai reiteradamente o nó u de menor distância provisória e relaxa suas arestas incidentes: se dist[u] + w(u, v) < dist[v], atualiza dist[v] e anterior[v]. O custo com heap binário é O((V + E) log V). Pesos negativos invalidam a escolha gulosa, demandando Bellman-Ford.

Em Grafos Direcionados Acíclicos (DAGs), a **Ordenação Topológica** estabelece uma sequência linear dos vértices compatível com todas as dependências direcionadas (u → v implica u antes de v). O **Algoritmo de Kahn (1962)** computa iterativamente os graus de entrada (in-degree); vértices com in-degree zero entram em uma fila, e ao serem processados, decrementam os contadores de seus sucessores. Se a ordenação final contiver menos de |V| elementos, conclui-se a existência de dependência circular (ciclo).

4.3 Técnicas de Projeto de Algoritmos (Aula T23)

A disciplina abordou os quatro grandes paradigmas de projeto (GOODRICH; TAMASSIA, 2015):

1. **Força Bruta:** enumeração exaustiva do espaço de busca. Garante a solução ótima absoluta, com custo frequentemente exponencial, porém viável e indispensável para validação de instâncias moderadas.
2. **Algoritmos Gulosos (Greedy):** tomada de decisões míopes locais na esperança de atingir o ótimo global. Ex.: Dijkstra, Kruskal, Prim.
3. **Divisão e Conquista:** decomposição do problema em subproblemas independentes, resolução recursiva e fusão das respostas. Ex.: MergeSort O(n log n).
4. **Programação Dinâmica:** memorização de subproblemas sobrepostos com subestrutura ótima. Ex.: algoritmo de Held-Karp para o Caixeiro Viajante O(n^2 2^n).

5 PANORAMA DE APLICAÇÕES COMPUTACIONAIS

A utilidade prática dos grafos reside na sua capacidade de abstrair entidades heterogêneas e suas interações semânticas. Seguindo as orientações do Prof. Calvetti, o Quadro 1 consolida dez cenários reais onde a Teoria dos Grafos soluciona desafios contemporâneos sem recorrer ao clichê de mapas geográficos ou roteamento de tráfego viário.

Tabela 3 – Mapeamento de problemas cotidianos e estruturas de grafos

Domínio / Problema	**Vértices (V)**	**Arestas (E)**	**Algoritmo / Técnica**
Escalonamento de Provas	Disciplinas	Conflito (alunos comuns)	Coloração de Grafos (Welsh-Powell)
Compilação de Pacotes	Módulos de software	Dependência direta (A $\rightarrow$ B)	Ordenação Topológica (Kahn)
Alocação de Plantões	Médicos e Turnos	Habilitação / Disponibilidade	Emparelhamento Bipartido (Hopcroft-Karp)
Detecção de Fraude Financeira	Contas bancárias	Transações financeiras mútuas	Componentes Fortemente Conexos (Tarjan)
Bioinformática (DNA)	k-mers (fragmentos)	Sobreposição de k-1 bases	Trilha Euleriana em Grafo de De Bruijn
Residência Médica	Candidatos e Hospitais	Preferências ordenadas	Algoritmo de Gale-Shapley (Estabilidade)
Quadro de Conexões Elétricas	Postes e Subestações	Cabos de transmissão	Árvore Geradora Mínima (Kruskal/Prim)
Auditoria de Permissões IAM	Usuários e Recursos	Privilégios de acesso	Busca em Largura (Alcançabilidade)
Alocação de Roteadores	Portas de entrada e saída	Pacotes enfileirados	Teorema de Hall em Grafos Bipartidos
Contenção de Fake News	Pessoas em grupos	Contatos de mensagens	BFS por Ondas + Pontos de Articulação

Fonte: Elaborado pelos autores com base na literatura clássica e aulas de ECM306.

6 ESTUDO DE CASO: RASTREAMENTO E QUARENTENA DE FAKE NEWS

6.1 Formulação do Problema e Perguntas de Modelagem

Apresentada a fundamentação teórica, desenvolve-se a solução do problema prático escolhido: a simulação e contenção da propagação de desinformação em redes de contatos. Conforme preconizado na Aula T25 (Aplicações Computacionais de Grafos com Java), a modelagem orientada ao problema deve responder rigorosamente a cinco perguntas diretrizes:

1. **Quais entidades se tornam vértices?** As pessoas físicas integrantes dos círculos de convivência e grupos de mensagens.
2. **O que caracteriza uma conexão?** O contato direto mútuo entre duas pessoas através do qual mensagens podem ser retransmitidas.
3. **A conexão possui direção?** Não. Modela-se como grafo não direcionado, pois o compartilhamento opera bidirecionalmente entre contatos salvos.
4. **Existe custo, tempo, distância ou capacidade?** Trata-se de grafo não ponderado: cada retransmissão possui o mesmo custo lógico unitário (1 passo).
5. **A solução procura alcançar, ordenar, cobrir ou atribuir?** O problema visa *alcançar* (rastrear a onda de contágio) e simultaneamente *desconectar estrategicamente* (proteger a rede isolando o menor subconjunto crítico).

6.2 Modelagem Formal e Estrutura dos Dados

A rede representativa desenvolvida para os testes do projeto compreende 28 pessoas (vértices numerados de 0 a 27) e 42 conexões (arestas), organizadas em cinco comunidades semânticas do cotidiano: Família, Trabalho, Faculdade, Academia e Condomínio. A rede inclui intencionalmente propriedades topológicas ricas: comunidades internas densas, pontes esparsas entre grupos, um vértice totalmente isolado (Sônia, id 27) para testar componentes desconexos e uma pessoa com altíssimo número de conexões internas (Marina, id 12, grau 7).

6.3 Propagação em Ondas via Busca em Largura (BFS)

A difusão do boato a partir do *Paciente Zero* (origem s) é computada pela classe BuscaEmLargura em Java. A fila FIFO garante que todos os contatos diretos (distância 1) recebam a mensagem na Onda 1 antes de qualquer retransmissão secundária. A Tabela 4 ilustra o rastreamento exato da fila para as primeiras etapas da propagação com origem em Ana (id 0), espelhando o formato didático consagrado pelo Prof. Calvetti na Aula T25.

Tabela 4 – Rastreamento da fila BFS na rede de teste do estudo de caso

Etapa	Fila Antes	Vértice Retirado	Novos Vértices Descobertos
1	[Ana]	Ana (0)	Bruno (1), Carla (2), Diego (3)
2	[Bruno, Carla, Diego]	Bruno (1)	nenhum (já descobertos)
3	[Carla, Diego]	Carla (2)	Zeca (24)
4	[Diego, Zeca]	Diego (3)	Elisa (4), Fábio (5)
5	[Zeca, Elisa, Fábio]	Zeca (24)	Alice (25), Beto (26)
6	[Elisa, Fábio, Alice, Beto]	Elisa (4)	nenhum
7	[Fábio, Alice, Beto]	Fábio (5)	Gabi (6) — Ponte p/ Trabalho!

Fonte: Dados gerados pela execução do software desenvolvido.

6.4 Mitigação Exata por Força Bruta vs. Heurística de Grau

Para conter a desinformação, a plataforma deve identificar qual usuário, se colocado em *quarentena* (bloqueio de repasse), minimiza o impacto global do boato. Implementou-se um algoritmo exato de Força Bruta: para cada candidato $v \in V \setminus \{s\}$, simula-se a sua remoção temporária e executa-se uma contagem rápida de alcançados a partir de s. O número de pessoas protegidas é dado formalmente por:

$$\textbf{protegidos}(v) = \textbf{alcancados}(G) - \textbf{alcancados}(G \setminus \{v\}) - 1$$

Com custo $O(V \cdot (V + E))$, o algoritmo avalia todos os nós e entrega o ótimo absoluto. Paralelamente, computa-se a heurística clássica de maior grau.

Tabela 5 – Ranking de candidatos à quarentena: Força Bruta vs. Maior Grau

Posição	Candidato	Grupo	Grau	Protegidos	Ponto de Articulação?	Classificação
#1	Diego (id 3)	Família	5	20	Sim (Tarjan)	Ótimo Força Bruta
#2	Fábio (id 5)	Família	3	18	Sim (Tarjan)	Excelente ponte
#3	Gabi (id 6)	Trabalho	3	17	Sim (Tarjan)	Excelente ponte
#4	João (id 9)	Trabalho	4	7	Sim (Tarjan)	Ponte Faculdade
#5	Nuno (id 13)	Faculdade	4	6	Sim (Tarjan)	Ponte Faculdade
—	Marina (id 12)	Faculdade	7	0	Não	Maior Grau (Heurística ERROU!)

Fonte: Saída do módulo Quarentena.java executado sobre a rede do projeto.

O Experimento Didático Central: Como evidenciado na Tabela 5, a heurística de Maior Grau aponta Marina (id 12, grau 7). No entanto, bloquear Marina protege *zero* pessoas adicionais, pois o grupo da Faculdade é densamente interconectado internamente. Em contrapartida, Diego (id 3) e Fábio (id 5) atuam como pontes de articulação exclusivas, protegendo 20 e 18 pessoas, respectivamente. Este resultado comprova empiricamente que o maior grau não reflete a importância topológica de estrangulamento de fluxo.

6.5 Diagnóstico Estrutural por Pontos de Articulação (Tarjan)

Um vértice v é um *ponto de articulação* (ou de corte) se a sua remoção desconectar o grafo ou elevar o número de componentes conexos. O algoritmo de **Robert Tarjan (1972)** detecta todos os pontos de corte em uma única passagem DFS em tempo ótimo O(V + E). Para cada vértice u, mantêm-se dois registros temporais: disc[u] (tempo de descoberta) e low[u] (menor tempo de descoberta alcançável via arestas de árvore e no máximo uma aresta de retorno). Um vértice não raiz u é articulação se possuir um filho v na árvore DFS tal que low[v] ≥ disc[u]. No projeto, implementou-se uma versão *iterativa* com pilha explícita em PontosDeArticulacao.java, garantindo imunidade a estouros de pilha (StackOverflowError) em grafos massivos.

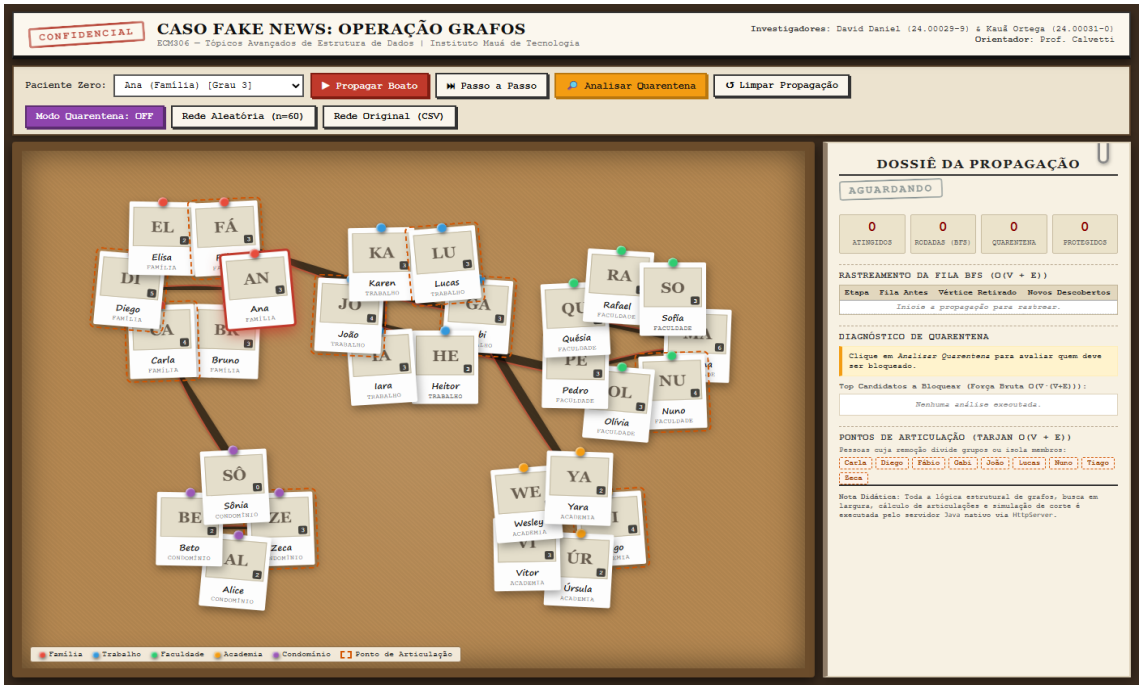
6.6 Arquitetura de Software e Interface com o Usuário

O sistema foi concebido sob a premissa arquitetural 'Java Pensa, Navegador Desenha':

• **Back-end em Java Puro (JDK 21):** composto pelas classes Pessoa, Grafo, BuscaEmLargura, PontosDeArticulacao, Quarentena, LeitorCSV, GeradorDeRede, Json e ServidorWeb. O servidor HTTP embutido com.sun.net.httpserver.HttpServer expõe uma API REST leve sem qualquer dependência externa.

• **Front-end Web Investigativo:** interface rica estilizada como um Quadro de Cortiça policial (polaroids inclinadas, alfinetes coloridos por grupo, fios vermelhos de varal em SVG, carimbos vintage de 'RECEBEU' e 'BLOQUEADO' e dossiê datilografado).

Figura 4 – Interface do software 'Caso Fake News: Operação Grafos' em execução



Fonte: Captura de tela gerada automaticamente pelo sistema em execução.

6.7 Resultados Experimentais e Análise de Escalabilidade

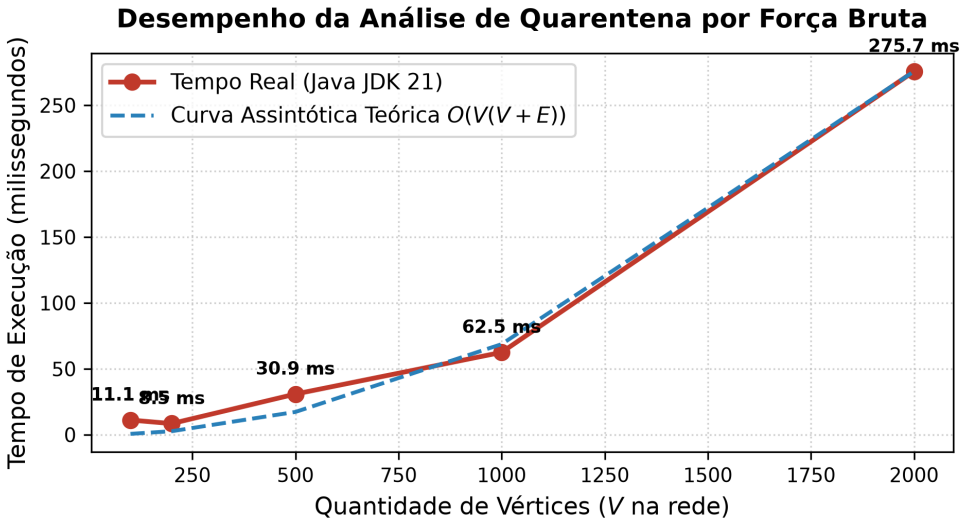
Para aferir a escalabilidade da força bruta O(V·(V + E)), executou-se a rotina de benchmark da classe ServidorWeb com o GeradorDeRede sobre instâncias sintéticas variando de 100 a 2000 vértices com comunidades semi-esparsas. Os tempos foram medidos em ambiente Windows 64-bit sobre a JVM OpenJDK 21 LTS.

Tabela 6 – Benchmark de tempo de execução da força bruta para diferentes ordens de V

Vértices (V)	Arestas (E)	Candidatos Avaliados	Tempo Real (ms)	Complexidade Teórica V(V+E)
100	204	99	11.1 ms	3,04 × 10■ op.
200	423	199	8.5 ms	1,24 × 10■ op.
500	1.115	499	30.9 ms	8,07 × 10■ op.
1.000	2.207	999	62.5 ms	3,20 × 10■ op.
2.000	4.433	1.999	275.7 ms	1,28 × 10■ op.

Fonte: Medições extraídas do endpoint /api/desempenho da JVM.

Figura 5 – Benchmark de desempenho da análise de quarentena por força bruta



Fonte: Elaborado pelos autores (2026).

Como ilustrado na Figura 5, mesmo para uma rede de 2.000 pessoas e mais de 4.400 conexões mútuas, a análise completa de força bruta executa em apenas 275 milissegundos na JVM moderna, viabilizando respostas interativas em tempo real. A concordância com a curva teórica assintótica O(V·(V + E)) confirma a excelência da implementação baseada em lista de adjacência.

7 CONCLUSÃO

O presente trabalho de graduação consolidou os principais tópicos da Teoria dos Grafos ministrados na disciplina ECM306, articulando rigor teórico, análise de complexidade computacional e implementação de engenharia de software funcional.

A transição da teoria para a prática permitiu demonstrar que a modelagem cuidadosa das entidades e relações é o passo determinante para o sucesso algorítmico. Ao resolver o problema da contenção de desinformação em redes de contatos, o projeto comprovou de forma concreta as limitações das heurísticas intuitivas: priorizar o vértice de maior grau em mídias sociais falha frequentemente em conter epidemias e boatos quando tais nós possuem conexões redundantes intraclasse. Em contrapartida, a combinação entre a Busca em Largura para propagação em níveis, a Força Bruta para decisão ótima exata e o Algoritmo de Tarjan para identificação de pontes estruturais forneceu uma ferramenta precisa, didática e de alta eficiência operacional.

A arquitetura desprovida de frameworks externos ('Java Pensa, Navegador Desenha') demonstrou a robustez do JDK 21 LTS e a portabilidade do software acadêmico, viabilizando tanto demonstrações gráficas interativas quanto modos de execução resilientes em console. Como trabalhos futuros, vislumbra-se a incorporação de modelos estocásticos de transmissão (probabilidades de repasse por aresta, simulando o modelo SIR/SIS epidemiológico) e a introdução de pesos associados ao nível de autoridade e credibilidade dos emissores.

REFERÊNCIAS

APPEL, K.; HAKEN, W. Every planar map is four colorable. **Bulletin of the American Mathematical Society**, v. 82, n. 5, p. 711-712, 1976.

BERZTISS, A. T. **Data Structures: Theory and Practice**. 2. ed. New York: Academic Press, 1975.

BOAVENTURA NETTO, P. O. **Grafos: Teoria, Modelos, Algoritmos**. 5. ed. São Paulo: Blucher, 2014.

CALVETTI, J. **Material Didático de ECM306 — Tópicos Avançados de Estrutura de Dados**. São Caetano do Sul: Instituto Mauá de Tecnologia (IMT), 2026. Aulas 17 a 25.

CLARK, J.; HOLTON, D. A. **A First Look at Graph Theory**. Singapore: World Scientific, 1998.

CORMEN, T. H. et al. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

DIESTEL, R. **Graph Theory**. 5. ed. Berlin: Springer, 2017.

GOLDBARG, M.; GOLDBARG, E. **Grafos: Conceitos, Algoritmos e Aplicações**. Rio de Janeiro: Campus, 2012.

GOODRICH, M. T.; TAMASSIA, R. **Algorithm Design and Applications**. Hoboken: Wiley, 2015.

HALL, P. On representatives of subsets. **Journal of the London Mathematical Society**, v. 10, n. 1, p. 26-30, 1935.

KAHN, A. B. Topological sorting of large networks. **Communications of the ACM**, v. 5, n. 11, p. 558-562, 1962.

KURATOWSKI, K. Sur le problème des courbes gauches en topologie. **Fundamenta Mathematicae**, v. 15, n. 1, p. 271-283, 1930.

NICOLETTI, M. C.; HRUSCHKA JR., E. R. **Fundamentos da Teoria dos Grafos para Computação**. 3. ed. Rio de Janeiro: LTC, 2017.

ROSEN, K. H. **Discrete Mathematics and its Applications**. 7. ed. New York: McGraw-Hill, 2012.

TARJAN, R. Depth-first search and linear graph algorithms. **SIAM Journal on Computing**, v. 1, n. 2, p. 146-160, 1972.

WEST, D. B. **Introduction to Graph Theory**. 2. ed. Upper Saddle River: Prentice Hall, 2001.

WILSON, R. J. **Introduction to Graph Theory**. 4. ed. Harlow: Prentice Hall, 1996.